

Montgomery Multiplication

Many algorithms in number theory, like [prime testing](#) or [integer factorization](#), and in cryptography, like RSA, require lots of operations modulo a large number. A multiplication like $x \cdot y \bmod n$ is quite slow to compute with the typical algorithms, since it requires a division to know how many times n has to be subtracted from the product. And division is a really expensive operation, especially with big numbers.

The **Montgomery (modular) multiplication** is a method that allows computing such multiplications faster. Instead of dividing the product and subtracting n multiple times, it adds multiples of n to cancel out the lower bits and then just discards the lower bits.

Montgomery representation

However the Montgomery multiplication doesn't come for free. The algorithm works only in the **Montgomery space**. And we need to transform our numbers into that space, before we can start multiplying.

For the space we need a positive integer $r \geq n$ coprime to n , i.e. $\gcd(n, r) = 1$. In practice we always choose r to be 2^m for a positive integer m , since multiplications, divisions and modulo r operations can then be efficiently implemented using shifts and other bit operations. n will be an odd number in pretty much all applications, since it is not hard to factorize an even number. So every power of 2 will be coprime to n .

The representative $\bar{x}$ of a number x in the Montgomery space is defined as:

$$\bar{x} := x \cdot r \bmod n$$

Notice, the transformation is actually such a multiplication that we want to optimize. So this is still an expensive operation. However you only need to transform a number once into the space. As soon as you are in the Montgomery space, you can perform as many operations as you want efficiently. And at the end you transform the final result back. So as long as you are doing lots of operations modulo n , this will be no problem.

Inside the Montgomery space you can still perform most operations as usual. You can add two elements ($x \cdot r + y \cdot r \equiv (x + y) \cdot r \bmod n$), subtract, check for equality, and even compute the greatest common divisor of a number with n (since $\gcd(n, r) = 1$). All with the usual algorithms.